

**IMPLEMENTASI SHORTEST PATH UNTUK OPTIMASI PENGIRIMAN  
PAKET ANTAR WILAYAH**

**LAPORAN**

Dosen Pengampu: Lia Silviana, S.TI., M.Kom.



Disusun oleh

Kelompok 8

|             |                              |           |
|-------------|------------------------------|-----------|
| Nama        | : 1. Sina Mahdi Sitanggang   | 251402008 |
|             | 2. Muhammad Ihsan Anwar      | 251402044 |
|             | 3. Muhammad Azkha Amorie     | 251402092 |
|             | 4. Fadila Lisma Sari         | 251402117 |
|             | 5. Qairsya Naurel Ein Yaliki | 251402120 |
|             | 6. Syifa Nazira              | 251402126 |
| Kelas       | : B                          |           |
| Mata Kuliah | : Struktur Data Algoritma    |           |

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI  
UNIVERSITAS SUMATERA UTARA  
2026**

## DAFTAR ISI

|                                                 |    |
|-------------------------------------------------|----|
| <b>DAFTAR ISI</b> .....                         | i  |
| <b>BAB I    PENDAHULUAN</b> .....               | 1  |
| 1.1   Latar Belakang.....                       | 1  |
| 1.2   Rumusan Masalah.....                      | 2  |
| 1.3   Tujuan.....                               | 2  |
| <b>BAB II   DASAR TEORI</b> .....               | 3  |
| 2.1   Graph .....                               | 3  |
| 2.2   Shortest Path .....                       | 3  |
| 2.3   Algoritma Dijkstra .....                  | 4  |
| 2.4   Priority Queue.....                       | 4  |
| <b>BAB III   ANALISIS DAN PERANCANGAN</b> ..... | 5  |
| 3.1   Flowchart .....                           | 5  |
| 3.2   Pseudocode .....                          | 6  |
| 3.3   Struktur Data.....                        | 7  |
| 3.3.1 Graph (Adjacency List).....               | 7  |
| 3.3.2 Array Distance List .....                 | 8  |
| 3.3.3 Array Parent .....                        | 8  |
| 3.3.4 Priority Queue .....                      | 8  |
| 3.3.5 Vector .....                              | 9  |
| <b>BAB IV   IMPLEMENTASI</b> .....              | 10 |
| 4.1   Potongan Kode Program.....                | 10 |
| 4.1.1 Struktur Data Edge .....                  | 10 |
| 4.1.2 Deklarasi Graph.....                      | 10 |
| 4.1.3 Input Ruas Jalan .....                    | 10 |
| 4.1.4 Inisialisasi Algoritma Dijkstra .....     | 10 |
| 4.1.5 Proses Algoritma Dijkstra .....           | 11 |
| 4.1.6 Rekonstruksi Rute .....                   | 11 |
| 4.1.7 Penyimpanan Hasil ke File.....            | 11 |
| 4.1.8 Visualisasi HTML .....                    | 11 |
| 4.2   Penjelasan Fungsi .....                   | 11 |

|                       |                                       |    |
|-----------------------|---------------------------------------|----|
| 4.2.1                 | Struktur Data Edge .....              | 11 |
| 4.2.2                 | Deklarasi Graph.....                  | 12 |
| 4.2.3                 | Input Ruas Jalan .....                | 12 |
| 4.2.4                 | Inisialisasi Algoritma Dijkstra ..... | 13 |
| 4.2.5                 | Proses Algoritma Dijkstra .....       | 13 |
| 4.2.6                 | Rekonstruksi Rute .....               | 13 |
| 4.2.7                 | Penyimpanan Hasil ke File.....        | 14 |
| 4.2.8                 | Visualisasi HTML .....                | 14 |
| <b>BAB V</b>          | <b>PENGUJIAN</b> .....                | 15 |
| 5.1                   | Contoh Input/Output.....              | 15 |
| 5.1.1                 | Input dan Output Terminal .....       | 15 |
| 5.1.2                 | Output File.....                      | 16 |
| 5.2                   | Screenshot Hasil .....                | 17 |
| 5.2.1                 | Output Visualisasi .....              | 17 |
| <b>BAB VI</b>         | <b>KESIMPULAN</b> .....               | 19 |
| 6.1                   | Kesimpulan .....                      | 19 |
| <b>DAFTAR PUSTAKA</b> | .....                                 | 21 |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Perkembangan teknologi informasi telah memberikan dampak besar terhadap berbagai bidang, termasuk dalam sistem distribusi dan pengiriman paket. Perusahaan logistik dituntut untuk mampu melakukan pengiriman secara cepat, tepat, dan efisien agar dapat memenuhi kebutuhan pelanggan. Dalam proses pengiriman paket antar wilayah, penentuan rute menjadi salah satu faktor penting karena berpengaruh terhadap waktu tempuh, biaya operasional, serta efektivitas distribusi barang.

Permasalahan yang sering terjadi dalam pengiriman paket adalah sulitnya menentukan jalur tercepat dari satu lokasi ke lokasi lainnya, terutama ketika terdapat banyak wilayah yang saling terhubung dengan jarak yang berbeda-beda. Oleh karena itu, diperlukan suatu metode yang mampu mencari jalur terpendek secara optimal sehingga proses distribusi dapat berjalan lebih efektif.

Salah satu algoritma yang dapat digunakan untuk menyelesaikan permasalahan tersebut adalah algoritma Dijkstra. Algoritma ini merupakan algoritma pencarian shortest path pada graf berbobot non-negatif yang bekerja dengan prinsip greedy untuk menentukan jarak minimum dari satu titik sumber ke titik tujuan lainnya. Dengan menggunakan algoritma Dijkstra, sistem dapat menentukan rute pengiriman paket yang paling efisien berdasarkan total jarak terkecil.

Pada project ini, algoritma Dijkstra diimplementasikan untuk optimasi pengiriman paket antar wilayah dengan menggunakan representasi graf berbobot. Setiap wilayah direpresentasikan sebagai simpul (node), sedangkan jalur antar wilayah direpresentasikan sebagai sisi (edge) yang memiliki bobot berupa jarak tempuh. Implementasi ini diharapkan mampu membantu proses pencarian rute tercepat sehingga pengiriman paket menjadi lebih optimal.

## **1.2 Rumusan Masalah**

1. Bagaimana cara merepresentasikan wilayah dan jalur pengiriman ke dalam bentuk graf berbobot?
2. Bagaimana implementasi algoritma Dijkstra untuk menentukan jalur terpendek pada proses pengiriman paket antar wilayah?
3. Bagaimana menampilkan hasil berupa jarak minimum dan rute pengiriman dari wilayah asal ke wilayah tujuan?
4. Bagaimana penggunaan struktur data priority queue dapat membantu meningkatkan efisiensi algoritma Dijkstra?

## **1.3 Tujuan**

1. Memahami konsep graf berbobot dalam pemodelan jalur pengiriman paket.
2. Mengimplementasikan algoritma Dijkstra untuk mencari jalur terpendek antar wilayah.
3. Menampilkan informasi rute tercepat beserta total jarak tempuh dari titik sumber ke titik tujuan.
4. Menganalisis penggunaan struktur data priority queue dalam meningkatkan performa pencarian shortest path.
5. Membuat sistem sederhana yang dapat membantu optimasi pengiriman paket antar wilayah secara lebih efisien.

## **BAB II**

### **DASAR TEORI**

#### **2.1 Graph**

Graph merupakan struktur data yang digunakan untuk merepresentasikan hubungan antar objek yang saling terhubung. Graph terdiri dari simpul (vertex/node) dan sisi (edge), di mana edge digunakan untuk menghubungkan satu node dengan node lainnya. Pada graph berbobot, setiap edge memiliki nilai bobot tertentu seperti jarak, waktu, atau biaya. Dalam implementasi sistem pengiriman paket, graph digunakan untuk memodelkan hubungan antar wilayah distribusi sehingga proses pencarian rute dapat dilakukan secara terstruktur dan efisien.

Menurut Djafar dan Faizal, “graf digunakan untuk merepresentasikan hubungan antar objek yang saling terhubung” sehingga cocok diterapkan pada permasalahan pencarian jalur terpendek. Pada project ini, setiap wilayah direpresentasikan sebagai node, sedangkan jalur penghubung antar wilayah direpresentasikan sebagai edge dengan bobot berupa jarak tempuh. Dengan menggunakan graph berbobot, sistem dapat menghitung total jarak perjalanan dan menentukan rute pengiriman yang lebih optimal.

#### **2.2 Shortest Path**

Shortest path merupakan metode pencarian lintasan dengan total bobot minimum dari suatu node sumber menuju node tujuan pada graph berbobot. Permasalahan shortest path banyak diterapkan pada sistem navigasi, transportasi, jaringan komputer, dan distribusi logistik karena mampu membantu menentukan rute paling efisien. Dalam sistem pengiriman paket, shortest path digunakan untuk menentukan jalur tercepat sehingga waktu pengiriman dan biaya operasional dapat diminimalkan.

Menurut Tamatjita dan Mahastama, shortest path adalah “proses pencarian lintasan minimum antara titik asal dan titik tujuan pada graf berbobot.” Pada project ini digunakan konsep single-source shortest path, yaitu pencarian jalur terpendek dari satu titik sumber ke seluruh titik tujuan lainnya.

### **2.3 Algoritma Dijkstra**

Algoritma Dijkstra merupakan algoritma greedy yang digunakan untuk mencari jalur terpendek pada graph berbobot non-negatif. Algoritma ini bekerja dengan memilih node yang memiliki jarak minimum sementara untuk diproses terlebih dahulu, kemudian memperbarui jarak menuju node tetangga apabila ditemukan jalur yang lebih pendek. Proses tersebut dilakukan secara berulang hingga seluruh node selesai diproses.

Menurut Sahid dkk., “algoritma Dijkstra mampu menghasilkan rute tercepat secara optimal pada graf berbobot non-negatif.” Algoritma ini banyak digunakan dalam sistem navigasi, jaringan komputer, dan optimasi distribusi barang karena mampu menghasilkan jalur minimum secara akurat. Pada project ini, algoritma Dijkstra digunakan untuk menentukan rute pengiriman paket antar wilayah dengan total jarak paling minimum sehingga proses distribusi menjadi lebih efisien.

### **2.4 Priority Queue**

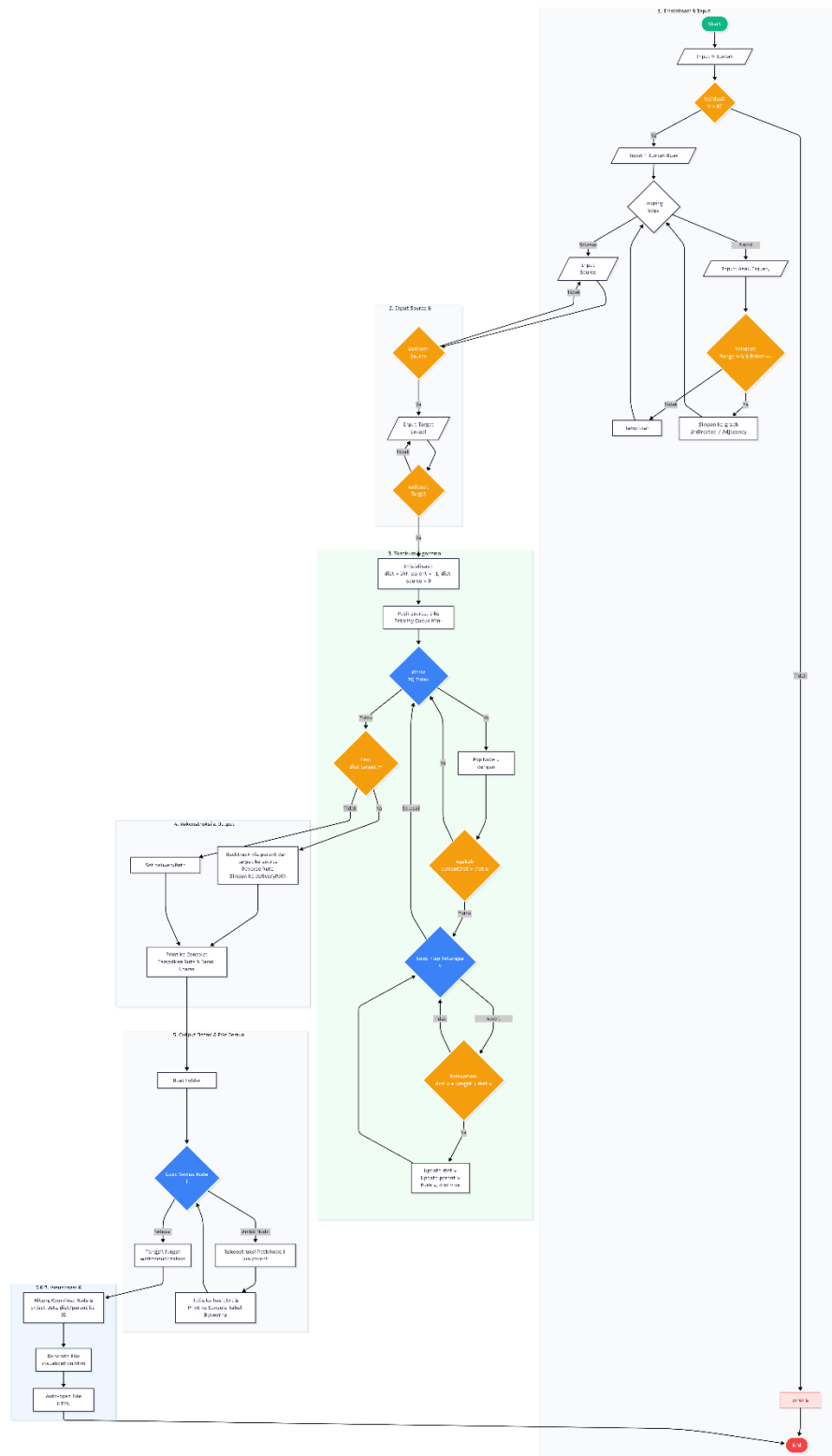
Priority Queue merupakan struktur data yang digunakan untuk menyimpan elemen berdasarkan prioritas tertentu. Dalam implementasi algoritma Dijkstra, priority queue digunakan untuk memilih node dengan jarak minimum secara lebih cepat dibandingkan pencarian biasa. Struktur data ini biasanya diimplementasikan menggunakan min heap sehingga proses pengambilan node prioritas dapat dilakukan dengan efisien.

Menurut Sahid dkk., penggunaan priority queue pada algoritma Dijkstra dapat meningkatkan efisiensi proses pencarian shortest path, terutama pada graph dengan jumlah node yang besar. Dengan adanya priority queue, algoritma tidak perlu melakukan pengecekan seluruh node secara berulang sehingga waktu eksekusi program menjadi lebih optimal. Oleh karena itu, priority queue sangat penting dalam implementasi algoritma shortest path pada sistem pengiriman paket antar wilayah.

# BAB III

## ANALISIS DAN PERANCANGAN

### 3.1 Flowchart





### 3.2 Pseudocode

Input:

V = jumlah lokasi  
E = jumlah ruas jalan  
source = lokasi awal  
target = lokasi tujuan

Output:

Jarak terpendek dan rute optimal

Deklarasi:

graph : Adjacency List  
dist[V] : array integer  
parent[V] : array integer  
pq : Priority Queue

Deskripsi:

1. Baca jumlah lokasi (V)
2. Baca jumlah ruas jalan (E)
3. Untuk setiap ruas jalan:  
    Input asal, tujuan, bobot  
    Tambahkan ke graph
4. Input lokasi awal (source)
5. Input lokasi tujuan (target)
6. Untuk setiap vertex i:  
     $\text{dist}[i] \leftarrow \infty$   
     $\text{parent}[i] \leftarrow -1$
7.  $\text{dist}[\text{source}] \leftarrow 0$
8. Masukkan (0, source) ke Priority Queue
9. Selama Priority Queue tidak kosong:  
    Ambil vertex u dengan jarak minimum  
    Untuk setiap tetangga v dari u:

```

    Jika  $\text{dist}[u] + \text{bobot}(u,v) < \text{dist}[v]$  maka
         $\text{dist}[v] \leftarrow \text{dist}[u] + \text{bobot}(u,v)$ 
         $\text{parent}[v] \leftarrow u$ 
        Masukkan  $(\text{dist}[v], v)$  ke Priority Queue
    EndIf
EndFor
EndWhile
10. Jika  $\text{dist}[\text{target}] = \infty$ 
    Tampilkan "Tujuan tidak terjangkau"
Else
    Rekonstruksi rute menggunakan array parent
    Tampilkan jarak terpendek
    Tampilkan rute optimal
EndIf
11. Simpan hasil ke file
12. Buat visualisasi graph dalam HTML

```

### 3.3 Struktur Data

Sistem optimasi pengiriman paket ini menggunakan beberapa struktur data untuk mendukung proses pencarian jalur terpendek dengan Algoritma Dijkstra.

#### 3.3.1 Graph (Adjacency List)

Graph digunakan untuk merepresentasikan hubungan antar lokasi pengiriman. Setiap lokasi direpresentasikan sebagai simpul (vertex), sedangkan ruas jalan direpresentasikan sebagai sisi (edge) yang memiliki bobot berupa jarak. Implementasi graph menggunakan Adjacency List karena lebih efisien dalam penggunaan memori dibandingkan Adjacency Matrix, terutama untuk graph yang jumlah ruas jalannya relatif sedikit dibandingkan jumlah simpul.

Struktur:

```

struct Edge {
    int to;
    int weight;
}

```

```
};
```

```
vector<vector<Edge>> graph;
```

Keterangan:

- to menyimpan tujuan vertex.
- weight menyimpan bobot atau jarak antar lokasi.
- graph menyimpan seluruh daftar ketetanggaan.

### 3.3.2 Array Distance (dist)

Array dist digunakan untuk menyimpan jarak terpendek dari lokasi sumber menuju seluruh lokasi pada graph.

Struktur:

```
vector<int> dist(V, INT_MAX);
```

Keterangan:

- Nilai awal setiap elemen adalah INT\_MAX (tak hingga).
- Setelah proses Dijkstra selesai, array ini berisi jarak terpendek ke setiap lokasi.

### 3.3.3 Array Parent

Array parent digunakan untuk menyimpan predecessor (simpul sebelumnya) dari setiap lokasi sehingga rute terpendek dapat direkonstruksi.

Struktur:

```
vector<int> parent(V, -1);
```

Keterangan:

- Nilai -1 menandakan tidak memiliki predecessor.
- Digunakan untuk membangun kembali jalur terpendek dari tujuan ke sumber.

### 3.3.4 Priority Queue

Priority Queue digunakan untuk memilih simpul dengan jarak terkecil secara otomatis pada setiap iterasi Algoritma Dijkstra.

Struktur:

```
priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>
> pq;
```

Keterangan:

- Elemen pertama menyimpan jarak.
- Elemen kedua menyimpan nomor vertex.
- Mendukung operasi pengambilan jarak minimum dengan kompleksitas yang efisien.

### 3.3.5 Vector

Struktur data vector digunakan untuk penyimpanan data yang bersifat dinamis, seperti daftar jalur pengiriman dan representasi graph.

Contoh:

```
vector<int> deliveryPath;
```

Keterangan:

- Menyimpan urutan lokasi yang dilalui pada jalur terpendek.
- Ukuran dapat bertambah secara dinamis sesuai kebutuhan program.

Dengan kombinasi Adjacency List, Priority Queue, Array Distance, dan Array Parent, Algoritma Dijkstra dapat dijalankan dengan kompleksitas waktu sebesar:

$$O((V+E)\log V)$$

V = jumlah vertex (lokasi),

E = jumlah edge (ruas jalan).

Kompleksitas tersebut membuat program mampu mencari jalur terpendek secara efisien pada jaringan distribusi pengiriman paket yang memiliki banyak lokasi dan jalur penghubung.

## BAB IV

### IMPLEMENTASI

#### 4.1 Potongan Kode Program

Pada bagian implementasi, program dibuat menggunakan bahasa C++ dengan menerapkan Algoritma Dijkstra untuk menentukan rute terpendek pengiriman paket antar wilayah. Program ini menggunakan beberapa library seperti `<iostream>`, `<vector>`, `<queue>`, `<climits>`, `<fstream>`, dan `<algorithm>`. Kode Program dapat diambil melalui link [github](https://github.com/MuhammadIhsanAnwar/TB-K8-SDA_Dijkstra) [https://github.com/MuhammadIhsanAnwar/TB-K8-SDA\\_Dijkstra](https://github.com/MuhammadIhsanAnwar/TB-K8-SDA_Dijkstra)

##### 4.1.1 Struktur Data Edge

```
struct Edge {  
    int to;  
    int weight;  
};
```

##### 4.1.2 Deklarasi Graph

```
vector<vector<Edge>> graph(V);
```

##### 4.1.3 Input Ruas Jalan

```
graph[u].push_back({v, w});  
graph[v].push_back({u, w});
```

##### 4.1.4 Inisialisasi Algoritma Dijkstra

```
vector<int> dist(V, INT_MAX);  
vector<int> parent(V, -1);  
priority_queue<pii, vector<pii>, greater<pii>> pq;  
  
dist[source] = 0;  
pq.push({0, source});
```

#### 4.1.5 Proses Algoritma Dijkstra

```
while (!pq.empty()) {
    int currentDist = pq.top().first;
    int u = pq.top().second;
    pq.pop();

    if (currentDist > dist[u])
        continue;

    for (auto edge : graph[u]) {
        int v = edge.to;
        int weight = edge.weight;

        if (dist[u] + weight < dist[v]) {
            dist[v] = dist[u] + weight;
            parent[v] = u;
            pq.push({dist[v], v});
        }
    }
}
```

#### 4.1.6 Rekonstruksi Rute

```
vector<int> deliveryPath;
if (dist[target] != INT_MAX) {
    int temp = target;
    while (temp != -1) {
        deliveryPath.push_back(temp);
        temp = parent[temp];
    }
    reverse(deliveryPath.begin(), deliveryPath.end());
}
```

#### 4.1.7 Penyimpanan Hasil ke File

```
string outFile = string("output\\") + "hasil.txt";
ofstream outputFile(outFile);
```

#### 4.1.8 Visualisai HTML

```
writeVisualization(graph, V, source, target, dist, parent);
```

### 4.2. Penjelasan Fungsi

#### 4.2.1 Struktur Data Edge

Struktur data edge digunakan untuk merepresentasikan hubungan antar lokasi pada graph. Setiap objek edge menyimpan dua informasi penting, yaitu node tujuan (to) dan bobot jarak (weight). Dengan adanya struktur ini, setiap

ruas jalan yang menghubungkan dua wilayah dapat disimpan secara teroganisir sehingga memudahkan proses pencarian jalur.

Pada sistem optimasi pengiriman paket, setiap ruas jalan dianggap sebagai hubungan antar wilayah yang memiliki jarak tertentu. Informasi tersebut kemudian digunakan oleh Algoritma Dijkstra untuk menghitung total jarak terpendek dari lokasi awal menuju lokasi tujuan.

#### **4.2.2 Deklarasi Graph**

Graph merupakan struktur utama yang digunakan dalam program untuk menggambarkan jaringan wilayah pengiriman paket. Implementasi graph menggunakan adjacency list yang disimpan dalam bentuk `vector<vector<edge>>`. Setiap indeks merepresentasikan satu lokasi, sedangkan isi vector tersebut berisi daftar lokasi lain yang terhubung secara langsung.

Penggunaan adjacency list dipilih karena lebih efisien dalam penggunaan memori dibandingkan adjacency matrix, terutama ketika jumlah ruas jalan tidak terlalu banyak dibandingkan jumlah lokasi. Selain itu, proses pencarian node yang terhubung juga menjadi lebih cepat dan mudah dilakukan.

#### **4.2.3 Input Ruas Jalan**

Bagian ini berfungsi untuk memasukkan data hubungan antar lokasi ke dalam graph. Pengguna diminta menginput lokasi asal, lokasi tujuan, serta bobot jarak yang menghubungkan kedua lokasi tersebut. Setelah data diterima, program akan menyimpan ke dalam adjacency list.

Karena graph yang digunakan bersifat tidak berarah (undirected graph), maka setiap ruas jalan disimpan pada kedua node yang saling terhubung. Dengan demikian, sistem dapat melakukan pencarian rute dari dua arah yang berbeda sesuai kebutuhan perhitungan jalur terpendek.

#### **4.2.4 Inisialisasi Algoritma Dijkstra**

Tahap inisialisasi dilakukan sebelum proses pencarian jalur dimulai. Pada tahap ini dibuat vector dist untuk menyimpan jarak terpendek sementara dari lokasi awal ke seluruh lokasi lainnya. Seluruh nilai diisi dengan INT\_MAX yang merepresentasikan nilai tak hingga atau belum terjangkau.

Selain itu, dibuat pula vector parent yang berfungsi untuk menyimpan node sebelumnya pada jalur terpendek. priority queue juga diinisialisasi untuk membantu proses pemilihan node dengan jarak terkecil. Lokasi awal kemudian diberi nilai jarak 0 karena merupakan titik awal pengiriman paket.

#### **4.2.5 Proses Algoritma Dijkstra**

Bagian ini merupakan inti dari program karena berisi implementasi Algoritma Dijkstra. Program akan mengambil node dengan nilai jarak terkecil dari priority queue, kemudian memeriksa seluruh node tetangga yang terhubung langsung dengannya.

Apabila ditemukan jalur baru yang memiliki jarak lebih pendek dibandingkan jarak sebelumnya, maka nilai jarak akan diperbarui dan node tersebut dimasukkan kembali ke dalam priority queue. Proses ini berlangsung secara berulang hingga seluruh node yang dapat dijangkau telah diproses. Hasil akhirnya adalah jarak terpendek dari lokasi awal menuju seluruh lokasi lainnya.

#### **4.2.6 Rekonstruksi Rute**

Setelah proses perhitungan selesai, program melakukan rekonstruksi jalur untuk mengetahui urutan lokasi yang harus dilalui. Rekonstruksi dilakukan menggunakan data yang tersimpan pada vector parent.

Program akan menelusuri node tujuan menuju node sumber dengan mengikuti parent dari masing-masing node. Karena proses penelusuran dilakukan dari belakang ke depan, maka urutan hasil perlu dibalik menggunakan fungsi reverse(). Hasil akhir berupa jalur lengkap dari lokasi awal menuju lokasi tujuan yang memiliki jarak paling pendek.



#### **4.2.7 Penyimpanan Hasil ke File**

Program menyediakan fitur penyimpanan hasil perhitungan ke dalam file teks dengan nama hasil.txt. File ini dibuat secara otomatis pada folder output setelah proses pencarian jalur selesai dilakukan.

Informasi yang disimpan meliputi lokasi tujuan, jarak terpendek yang diperoleh, serta rute yang harus dilalui untuk mencapai lokasi tersebut. Fitur ini memudahkan pengguna untuk mendokumentasikan hasil perhitungan tanpa harus mencatat ulang hasil yang ditampilkan pada layar.

#### **4.2.8 Visualisasi HTML**

Visualisasi dibuat menggunakan fungsi writeVisualization(). Fungsi ini menghasilkan file HTML yang berisi tampilan graph interaktif lengkap dengan node, edge, bobot jarak, serta jalur terpendek yang ditemukan oleh Algoritma Dijkstra.

Pada visualisasi tersebut, lokasi awal, lokasi tujuan, dan rute aktif ditampilkan menggunakan warna yang berbeda sehingga mudah dibedakan oleh pengguna. Selain itu, terdapat animasi pergerakan paket yang menggambarkan proses pengiriman dari lokasi awal menuju lokasi tujuan. Visualisasi ini membantu pengguna memahami hasil perhitungan secara lebih intuitif dibandingkan hanya melihat data dalam bentuk teks.

## BAB V

### PENGUJIAN

#### 5.1 Contoh Input/Output

Pengujian sistem dilakukan dengan menggunakan dataset sederhana untuk memverifikasi kebenaran algoritma, keakuratan perhitungan jarak, serta fungsionalitas input/output pada terminal dan file eksternal. Skenario pengujian menggunakan 4 lokasi (A, B, C, D) dengan 5 ruas jalan yang saling terhubung.

##### 5.1.1 Input dan Output Terminal

###### a. Input Graf

Program menerima 4 lokasi (A, B, C, D) dan 5 ruas jalan yang membentuk graf berbobot tidak berarah. Setiap edge disimpan dalam adjacency list untuk efisiensi pencarian.

```
=====
SISTEM OPTIMASI PENGIRIMAN PAKET
ALGORITMA DIJKSTRA
=====

Program mencari jalur terpendek dari lokasi awal ke tujuan pengiriman paket.
Semua titik lokasi di peta akan diberi huruf: A, B, C, ...

Masukkan jumlah titik lokasi di peta: 4
(Misal 4 berarti lokasi: A, B, C, D)

Masukkan jumlah ruas jalan yang menghubungkan lokasi: 5

Masukkan data ruas jalan antar lokasi
Gunakan huruf A..D untuk semua lokasi.
Setiap ruas jalan akan diminta dalam 3 input terpisah:
- Lokasi Asal : huruf lokasi awal
- Lokasi Tujuan: huruf lokasi akhir
- Bobot Jarak : angka jarak/berat

Masukkan data ruas jalan ke-1:
Lokasi Asal: A
Lokasi Tujuan: B
Bobot Jarak: 10
Ruas jalan berhasil ditambahkan!

Masukkan data ruas jalan ke-2:
Lokasi Asal: A
Lokasi Tujuan: C
Bobot Jarak: 4
Ruas jalan berhasil ditambahkan!

Masukkan data ruas jalan ke-3:
Lokasi Asal: B
Lokasi Tujuan: C
Bobot Jarak: 3
Ruas jalan berhasil ditambahkan!

Masukkan data ruas jalan ke-4:
Lokasi Asal: C
Lokasi Tujuan: D
Bobot Jarak: 8
Ruas jalan berhasil ditambahkan!

Masukkan data ruas jalan ke-5:
Lokasi Asal: B
Lokasi Tujuan: D
Bobot Jarak: 15
Ruas jalan berhasil ditambahkan!
```

b. Source dan Target

Lokasi awal ditetapkan sebagai A dan tujuan sebagai D. Algoritma Dijkstra akan mencari jarak minimum dari A ke semua node, dengan fokus pada pencapaian node D.

```
Masukkan lokasi awal pengiriman (huruf): A
Masukkan lokasi tujuan pengiriman paket (huruf): D

=====
HASIL PERHITUNGAN RUTE TERPENDEK
=====

Rute optimal pengiriman paket dari A ke D:
Jarak: 12 Km
Rute: A -> C -> D
```

c. Detail Rute

Program berhasil membaca seluruh input graf, menjalankan algoritma Dijkstra, dan menampilkan jarak minimum serta rute terpendek dari lokasi awal A ke lokasi tujuan D. Hasil perhitungan menunjukkan bahwa jalur  $A \rightarrow C \rightarrow D$  dengan total jarak 12 merupakan rute tercepat, yang sesuai dengan prinsip shortest path pada graf berbobot non-negatif. Kemudian program akan secara otomatis membuat file visualisasi HTML dan menyimpan hasil lengkap ke hasil.txt untuk dokumentasi.

```
=====
DETAIL RUTE KE SEMUA LOKASI
=====

Lokasi | Jarak Terpendek | Rute
-----|-----|-----
A      | 0                | A
-----|-----|-----
B      | 7                | A -> C -> B
-----|-----|-----
C      | 4                | A -> C
-----|-----|-----
D      | 12               | A -> C -> D
-----|-----|-----

Visualisasi HTML berhasil dibuat dan dibuka di browser!
Hasil perhitungan juga telah disimpan ke file output\hasil.txt
```

### 5.1.2 Output File

Setelah proses hitungan selesai, program secara otomatis menyimpan seluruh hasil ke dalam file teks bernama hasil.txt pada folder output/. File ini berisi rekap jarak terpendek dan rute optimal ke seluruh lokasi yang dapat

dijangkau dari sumber. Berikut ini adalah isi file hasil.txt berdasarkan data pengujian:

```
HASIL RUTE TERPENDEK

Tujuan Lokasi A
Jarak Terpendek : 0
Rute             : A
-----

Tujuan Lokasi B
Jarak Terpendek : 7
Rute             : A C B
-----

Tujuan Lokasi C
Jarak Terpendek : 4
Rute             : A C
-----

Tujuan Lokasi D
Jarak Terpendek : 12
Rute             : A C D
-----
```

File ini berfungsi sebagai dokumentasi eksternal yang dapat dibuka ulang kapan saja tanpa perlu menjalankan program, serta memudahkan verifikasi hasil secara manual atau untuk keperluan pelaporan.

## 5.2 Screenshot Hasil

Dokumentasi visual berupa tangkapan layar (screenshot) diambil untuk membuktikan bahwa program berjalan sesuai dengan perancangan dan menghasilkan keluaran yang dapat diamati langsung oleh pengguna. Screenshot mencakup tampilan terminal saat eksekusi serta hasil visualisasi interaktif yang dihasilkan oleh sistem.

### 5.2.1 Output Visualisasi

Visualisasi dibuat secara otomatis dalam format HTML menggunakan library vis-network. Tampilan ini menampilkan graf interaktif dengan fitur berikut:

- a. Node berwarna

Lokasi awal (hijau/teal), lokasi tujuan (merah), node biasa (biru), dan rute aktif (kuning).

b. Edge Berbobot

Garis penghubung menampilkan nilai jarak tempuh.

c. Animasi paket

Simbol pergerakan paket yang menelusuri jalur terpendek secara real-time.

d. Dropdown Interaktif

Pengguna dapat mengganti tujuan pengiriman dan melihat perubahan rute secara dinamis.



## **BAB VI**

### **KESIMPULAN**

#### **6.1 Kesimpulan**

Berdasarkan hasil analisis, perancangan, implementasi, dan pengujian yang telah dilakukan pada program Implementasi Shortest Path untuk Optimasi Pengiriman Paket Antar Wilayah, dapat disimpulkan bahwa sistem yang dibangun telah berhasil memenuhi tujuan yang ditetapkan. Program mampu merepresentasikan hubungan antar wilayah dalam bentuk graph, di mana setiap wilayah direpresentasikan sebagai node dan setiap ruas jalan direpresentasikan sebagai edge yang memiliki bobot jarak tertentu. Representasi tersebut memungkinkan proses pencarian jalur dilakukan secara terstruktur dan sistematis.

Algoritma Dijkstra yang diterapkan pada program terbukti mampu menentukan jalur terpendek dari lokasi awal menuju lokasi tujuan dengan mempertimbangkan bobot jarak pada setiap ruas jalan. Hasil pengujian menunjukkan bahwa program dapat menghitung jarak minimum secara akurat, menampilkan urutan rute yang harus dilalui, serta memberikan informasi jarak terpendek ke seluruh lokasi yang terhubung dengan titik awal pengiriman. Dengan demikian, algoritma yang digunakan dapat membantu proses pengambilan keputusan dalam menentukan jalur pengiriman yang lebih efisien.

Selain menghasilkan output dalam bentuk teks pada terminal, program juga mampu menyimpan hasil perhitungan ke dalam file sehingga data dapat didokumentasikan dan digunakan kembali apabila diperlukan. Program juga dilengkapi dengan fitur validasi input yang membantu mengurangi kesalahan pengguna saat memasukkan data lokasi maupun bobot jarak. Hal ini menunjukkan bahwa sistem tidak hanya berfungsi dengan baik dari sisi perhitungan, tetapi juga memberikan kemudahan dalam proses penggunaan.

Visualisasi graph berbasis HTML yang dihasilkan menjadi nilai tambah dari sistem yang dibangun. Melalui visualisasi tersebut, pengguna dapat melihat hubungan antar lokasi, jalur terpendek yang dipilih, serta animasi proses pengiriman paket secara lebih jelas dan interaktif. Visualisasi ini membantu

pengguna memahami hasil perhitungan Algoritma Dijkstra dengan lebih mudah dibandingkan hanya melihat data dalam bentuk teks.

Secara keseluruhan, program yang dikembangkan telah berhasil mengimplementasikan konsep shortest path menggunakan Algoritma Dijkstra untuk mengoptimalkan pengiriman paket antar wilayah. Sistem mampu memberikan solusi pencarian rute yang efektif, efisien, dan mudah dipahami, sehingga dapat menjadi salah satu alternatif dalam membantu proses perencanaan jalur distribusi barang pada suatu jaringan wilayah.

## DAFTAR PUSTAKA

- Djafar, I., & Faizal. 2018. “*Single-Source Shortest Path pada Graf Berbobot Menggunakan Algoritma Dijkstra dan Bellman-Ford.*” Seminar Ilmiah Sistem Informasi dan Teknologi Informasi (SISITI), hlm. 144–152.
- Riti, Y. F., Iskandar, J. S., & Hendra. 2023. “*Comparison Analysis of Graph Theory Algorithms for Shortest Path Problem.*” Jurnal Sisfokom, Vol. 12, No. 3, hlm. 517–524.
- Sahid, S., Pakpahan, M. A., Winanda, R. P., Lubis, M. R., & Perdana, A. 2026. “*Implementasi Algoritma Shortest Path untuk Optimasi Rute pada Sistem Navigasi Lokasi.*” Bridge : Jurnal Publikasi Sistem Informasi dan Telekomunikasi, Vol. 2, No. 1.
- Tamatjita, E. N., & Mahastama, A. W. 2016. “*Shortest Path with Dynamic Weight Implementation using Dijkstra’s Algorithm.*” ComTech: Computer, Mathematics and Engineering Applications, Vol. 7, No. 3, hlm. 223–231.
- Zaki, A. 2017. “*Algoritma Dijkstra : Teori dan Aplikasinya.*” Jurnal Matematika UNAND, Vol. 6, No. 4, hlm. 1–8.